

Multidimensional Index-based For Each

Document #: P4150R0 [[Latest](#)] [[Status](#)]
Date: 2026-04-16
Project: Programming Language C++
Audience: Library Evolution
Reply-to: Nicolas Morales
<nmmoral@sandia.gov>
Mark Hoemmen
<mhoemmen@nvidia.com>
Bryce Adelstein Lelbach
<blelbach@nvidia.com>
Damien Lebrun-Grandie
<lebrungrandt@ornl.gov>

Contents

- [1 Revision History](#)
 - [1.1 P4150R0: 2026-4 Post-Croydon Mailing](#)
- [2 Abstract](#)
- [3 Motivation and Background](#)
 - [3.1 Stencil operations](#)
 - [3.2 Iteration over a partition of an mdspan](#)
 - [3.3 Current practice](#)
 - [3.3.1 Ranges: `views::cartesian\_product` of `views::indices`](#)
 - [3.3.2 Using Ranges for this is hard](#)
 - [3.3.3 Changing the iteration order is harder](#)
 - [3.3.4 Non-Standard approaches](#)
- [4 Proposed Interface](#)
 - [4.1 Iteration order](#)
- [5 Design Considerations](#)
 - [5.1 Should there be an integer overload for the rank-1 case?](#)
 - [5.2 Should there be extents overloads?](#)
 - [5.3 Why permit custom layouts?](#)

- 5.4 Why not iterate over slices?
- 5.5 Header name
- 5.6 Alternative Names
- 5.7 Should we make layout mappings and extents ranges and just use `ranges::for_each`?
- 5.8 Does this supersede the need for a `for_each` (element) algorithm for `mdspan`?
- 5.9 Do we want to return anything?
 - 5.9.1 No need to return an “iterator” analog
 - 5.9.2 No need to return the function object
- 6 Proposed Wording
 - 6.1 Add `__cpp_lib_for_each_index` macro
 - 26.?? Header `<mdalgorithm>` synopsis
 - 26.?? Multidimensional algorithms
 - 26.??1 For each index
- 7 Acknowledgments
- 8 References

§ 1 Revision History

§ 1.1 P4150R0: 2026-4 Post-Croydon Mailing

- Initial paper revision

§ 2 Abstract

Before	After
<pre>std::ranges::for_each(std::execution::par_unseq, std::views::cartesian_product(std::views::indices(A.extent(0)), std::views::indices(A.extent(1))), [=] (auto idx) { auto [i, j] = idx; B[j, i] = A[i, j]; });</pre>	<pre>std::for_each_index(std::execution::par_unseq, A.mapping(), [=] (auto i, auto j) { B[j, i] = A[i, j]; });</pre>

§ 3 Motivation and Background

C++23’s introduction of `mdspan` ([P0009R18]) opens the question of how to invoke a callable on each element of a given `mdspan`. That is, what should the `mdspan` analog of “`ranges::for_each`” be? There are at least two options.

1. Iterate over the domain of an `mdspan`, that is, its extents.
2. Iterate over the range of an `mdspan`, that is, its elements.

This paper proposes (1). (2) is left for another future paper.

Some users may ask why we would want to iterate over the extents of an `mdspan` instead of over the elements directly, as `ranges::for_each` on a `span` would do. The following examples show why.

§ 3.1 Stencil operations

Many algorithms that perform an operation on every element of a multidimensional array do not just look at the “current element” of the array. They also need to look at “neighboring” elements, or otherwise include elements whose multidimensional indices are a function of the current element’s multidimensional index. One calls this genre of algorithms “stencil operations.” Examples include

- the finite-difference method for approximating the solution of partial differential equations; and
- convolution operations in image processing, that compute each element of the result using a “window” surrounding each pixel in the input image.

For example, the following code computes the Laplacian (a second-order derivative approximation) `delta_u[r, c]` of a function `u` evaluated at discrete points `r, c` on a regular grid, where `h` is the spatial difference between sample points in each dimension.

```
for (int r = 1; r < u.extent(0) - 1; ++r) {
    for (int c = 1; c < u.extent(1) - 1; ++c) {
        delta_u[r-1, c-1] =
            h*h * (u[r+1, c] + u[r-1, c] + u[r, c+1] + u[r, c-1] - 4.0 * u[r,
                c]);
    }
}
```

Trying to express this as an operation over elements would require computing pointer offsets by hand. That, in turn, would assume that the user knows that they have a strided layout, and that the accessor is `default_accessor` or something like it.

```
auto extents_range = std::views::cartesian_product(
    std::views::iota(decltype(u)::index_type(1), u.extent(0)-1),
    std::views::iota(decltype(u)::index_type(1), u.extent(1)-1));
auto u_range = extents_range | std::views::transform(
    [u] (auto pair) { auto [r, c] = pair; return u[r, c]; });
auto delta_u_range = extents_range | std::views::transform(
    [delta_u] (auto pair) { auto [r, c] = pair; return delta_u[r, c]; });
```

```

auto loop_body = [h,u] (auto& delta_u_rc, const auto& u_rc) {
    const auto& u_rp1_c = &u_rc + u.stride(0);
    const auto& u_rm1_c = &u_rc - u.stride(0);
    const auto& u_r_cp1 = &u_rc + u.stride(1);
    const auto& u_r_cm1 = &u_rc - u.stride(1);
    delta_u_rc = h*h * (u_rp1_c + u_rm1_c + u_r_cp1 + u_r_cm1 - 4.0 * u_rc);
};
std::ranges::transform(u_range, delta_u_range, loop_body);

```

Asking users to do this would defeat two key design features of `mdspan`:

1. genericity over the layout and accessor, and
2. the familiar multidimensional array syntax.

§ 3.2 Iteration over a partition of an `mdspan`

Sometimes algorithms want to iterate over a partition of an `mdspan` into slices. Three examples come to mind.

First, some extents might have a meaning separate from that of other extents, so it makes sense to iterate over a subset of extents. For example, a rank-3 `mdspan` with extents `<int, dynamic_extent, dynamic_extent, 3>` might represent a collection of 3-D points (or a tensor field) in 2-D space. Each slice `submdspan(x, r, c, full_extent)` has the three values at spatial coordinate `r, c`.

```

template<class Layout, class Accessor>
void iterate(mdspan<float,
    extents<int, dynamic_extent, dynamic_extent, 3>,
    Layout, Accessor> x)
{
    for (int r = 0; r < x.extent(0); ++r) {
        for (int c = 0; c < x.extent(1); ++c) {
            auto x_p = submdspan(x, r, c, full_extent);
            /* ... code that expects a rank-1 mdspan ... */
            point3d p{.x = x_p[0], .y = x_p[1], .z = x_p[2]};
            /* ... code that expects point3d ... */
        }
    }
}

```

Rewriting this as an iteration over the elements of an `mdspan` would again call for breaking the layout abstraction, for example by assuming that the three elements in a “point” are stored contiguously in memory with no padding between “points.”

```

auto extents_range = std::views::cartesian_product(
    std::views::indices(u.extent(0)),
    std::views::indices(u.extent(1)));
auto x_points_range = extents_range | std::views::transform(
    [u] (auto pair) { auto [r, c] = pair; return x[r, c, 0]; });

auto loop_body = [x] (auto& x_rc0) {

```

```

auto* x_rc0_ptr = &x_rc0;
/* ... code that expects a contiguous array ... */
point3d p{.x = x_rc0[0], .y = x_rc0[1], .z = x_rc0[2]};
/* ... code that expects point3d ... */
};
std::ranges::for_each(x_points_range, loop_body);

```

Second, users might have supplied data in a “structure of arrays” format, and we want to transform it in place into an “array-of-structures” format. The `mdspan` over which we want to iterate doesn’t actually exist.

```

struct arrays {
    mdspan<float, dims<2>> x;
    mdspan<float, dims<2>> y;
    mdspan<float, dims<2>> z;
};

void iterate(arrays xyz) {
    for (int r = 0; r < x.extent(0); ++r) {
        for (int c = 0; c < x.extent(1); ++c) {
            point3d p{
                .x = xyz.x[r, c],
                .y = xyz.y[r, c],
                .z = xyz.z[r, c]};
            /* compute with code that expects point3d */
        }
    }
}

```

Third, our algorithm might need to partition the `mdspan` in order to reveal independent work. For example, a rank-2 Fast Fourier Transform (FFT) can be computed by performing rank-1 FFTs independently on first the leftmost extent, and then the rightmost extent.

§ 3.3 Current practice

§ 3.3.1 Ranges: `views::cartesian_product` of `views::indices`

The C++ Standard Library already has the tools we need in order to iterate over either the domain or the range of an `mdspan`. Iterating over the domain of an `mdspan` `x` would be equivalent to `ranges::for_each` over the range `extents_range(x.extents())` as given below. (We omit the possibility of changing the index order for now.)

```

template<class IndexType, size_t... Extents>
constexpr auto
extents_range(std::extents<IndexType, Extents...> e) {
    return [e] <size_t... Indices> (std::index_sequence<Indices...>) {
        return std::views::cartesian_product(
            std::views::indices(e.extent(Indices))...
        );
    } (std::make_index_sequence<sizeof...(Extents)>());
}

```

[P3060R3] introduced the `views::indices` feature in C++26.

Iterating over the range of `x` would be equivalent to `ranges::for_each` over the range `elements_range(x)` as given below. (Again, we omit the possibility of changing the index order for now, as well as any optimizations for particular layouts or accessors.)

```
template<class ElementType, class Extents, class Layout, class Accessor>
constexpr auto
elements_range(std::mdspan<ElementType, Extents, Layout, Accessor> x) {
    return extents_range(x.extents()) | std::views::transform(
        [x] (auto index_tuple) {
            auto [...indices] = index_tuple;
            return x[indices...];
        }
    );
}
```

We could combine these two techniques into a multidimensional analog of `views::enumerate` (see [\[P2164R9\]](#), which was adopted in C++23).

```
template<class ElementType, class Extents, class Layout, class Accessor>
constexpr auto
enumerate_range(std::mdspan<ElementType, Extents, Layout, Accessor> x) {
    return std::views::zip(extents_range(x.extents()), elements_range(x));
}
```

§ 3.3.2 Using Ranges for this is hard

Using Ranges to iterate over an `mdspan` is verbose and hard to write.

1. Users have to remember at least two function names, one of which involves a Latinate spelling of a French philosopher.
2. `views::cartesian_product` gives the indices as a range of tuples. However, `mdspan::operator[]` does not accept indices as a tuple. Thus, users must either decompose each tuple into indices (either with structured binding or `std::get`), or use `std::apply` with a lambda to invoke the `mdspan`'s `operator[]`.
3. If users write the above `extents_range` or `elements_range` functions, they must remember to capture the extents or `mdspan` object by value. Otherwise, the returned range may dangle.

§ 3.3.3 Changing the iteration order is harder

Performance often depends on the ability to control iteration order. Using the above Ranges approach, but changing the iteration order from rightmost-extent-first (row-major) to leftmost-extent-first (column-major), is a difficult C++ exercise. We show the result below. It comes from a [pull request here](#). The code is hard to read, though perhaps a more clever C++ programmer could make it a bit shorter. It also has a higher function call depth and might otherwise hinder compilers' ability to optimize.

```
template<class Callable, class IndexType, size_t Extent>
void for_each_one_extent(Callable&& callable,
```

```

    extents<IndexType, Extent> ext)
{
    if constexpr (ext.static_extent(0) == dynamic_extent) {
        for (IndexType index = 0; index < ext.extent(0); ++index) {
            std::forward<Callable>(callable)(index);
        }
    }
    else {
        [&<size_t ... Indices> (index_sequence<Indices...>) {
            (std::forward<Callable>(callable)(Indices), ...);
        } (make_index_sequence<Extent>());
    }
}

template<class Callable, class IndexType, size_t ... Extents,
        class ExtentsReorderer, class ExtentsSplitter, class IndicesReorderer>
void for_each_in_extents_impl(Callable&& callable,
    extents<IndexType, Extents...> ext,
    ExtentsReorderer reorder_extents,
    ExtentsSplitter split_extents,
    IndicesReorderer reorder_indices)
{
    if constexpr(ext.rank() == 0) {
        return;
    } else if constexpr(ext.rank() == 1) {
        for_each_one_extent(callable, ext);
    } else {
        // 1. Reorder the input extents.
        auto reordered_extents = reorder_extents(ext);

        // 2. Split into "left" and "right."
        //     For row-major and column-major, the resulting left_exts
        //     should always have rank 1 (i.e., only contain one extent).
        auto [left_exts, right_exts] = split_extents(reordered_extents);

        // 3. Create a lambda that loops over the right extents,
        //     and takes the left extent(s) as input.
        auto inner = [&] (auto... left_indices) {
            auto next = [&] (auto... right_indices) {
                // 4. "Fix" the order of indices to match
                //     the above reordering of extents.
                std::apply(std::forward<Callable>(callable),
                    reorder_indices(left_indices..., right_indices...));
            };
            for_each_in_extents_impl(next, right_exts, reorder_extents,
                split_extents, reorder_indices);
        };

        // 5. Take the above lambda and loop over the left extent(s).
        for_each_in_extents_impl(inner, left_exts, reorder_extents,
            split_extents, reorder_indices);
    }
}

auto extents_identity = [<class IndexType, size_t ... Extents>(
    extents<IndexType, Extents...> ext)

```

```

{
    return ext;
};

auto extents_reverse = []<class IndexType, size_t ... Extents>(
    extents<IndexType, Extents...> ext)
{
    constexpr size_t N = sizeof...(Extents);
    return [&<size_t ... Indices>(index_sequence<Indices...>) {
        return extents<
            IndexType,
            decltype(ext)::static_extent(N - 1 - Indices)...
        >{ext.extent(N - 1 - Indices)...};
    } (make_index_sequence<N>());
};

// Return a parameter pack as a tuple.
auto indices_identity = [](auto... indices) {
    return tuple{indices...};
};

// Return the reverse of a parameter pack as a std::tuple.
auto indices_reverse = [](auto... args) {
    constexpr std::size_t N = sizeof...(args);
    return [&<size_t ... Indices>(index_sequence<Indices...>) {
        return tuple{(args...[N - 1 - Indices])...};
    } (make_index_sequence<N>());
};

// Row-major iteration
template<class Callable, class IndexType, size_t ... Extents>
void for_each_in_extents(Callable&& callable,
    extents<IndexType, Extents...> ext,
    layout_right)
{
    for_each_in_extents_impl(std::forward<Callable>(callable), ext,
        extents_identity, split_extents_at_leftmost, indices_identity);
}

// Column-major iteration
template<class Callable, class IndexType, size_t ... Extents>
void for_each_in_extents(Callable&& callable,
    extents<IndexType, Extents...> ext,
    layout_left)
{
    for_each_in_extents_impl(std::forward<Callable>(callable), ext,
        extents_reverse, split_extents_at_rightmost, indices_reverse);
}

```

§ 3.3.4 Non-Standard approaches

Many non-Standard C++ approaches loop over multidimensional indices. Compiler extensions like [\[OpenACC\]](#) and [\[OpenMP\]](#) let the user annotate their nested `for` loops. Both OpenACC and OpenMP have annotations specifically for multidimensional loops, such as

flattening (condensing multiple loop levels into a single level for more straightforward parallelization) and tiling (reorganizing loop order into small compile-time-sized “tiles”).

C++ also has library-based solutions. The Kokkos performance-portability framework provides multidimensional loop iteration via `MDRangePolicy` overloads of its `parallel_for` algorithm [Kokkos]. NVIDIA’s CUB (“Cuda UnBound”) offers parallel algorithms for iterating over an `mdspan`’s extents (`cub::DeviceFor::ForEachInExtents`) and for doing so with an ordering hint given by the `mdspan`’s layout mapping (`cub::DeviceFor::ForEachInLayout`). HPX, which provides a parallel library that closely mirrors the standard C++ library, has extensions for index-based multidimensional loops [HPX].

Both compiler extensions and library solutions give users ways to control iteration order.

§ 4 Proposed Interface

In short, we would like to propose `for_each_index`, an algorithm that enables single and multi-dimensional index-based loops.

Taking the above matrix transpose example, the usage would look something like this:

```
std::for_each_index(std::execution::par_unseq,
    A.mapping(),
    [=] (auto i, auto j) {
        B[j, i] = A[i, j];
    });
```

We propose parallel and non-parallel overloads of `for_each_index`. All overloads take a layout mapping, and iterate over the mapping’s extents. The layout mapping is only a hint. Implementations are encouraged to use it to choose iteration order and parallelization strategy for best performance, for example by respecting spatial locality.

```
template<class Mapping, class Fun>
constexpr void for_each_index(const Mapping& mapping, Fun fun);

template<class ExecutionPolicy, class Mapping, class Fun>
void for_each_index(ExecutionPolicy&& policy, const Mapping& mapping, Fun
    fun);
```

We permit the mapping to have rank-1 or even rank-0 extents. This makes it easier to write rank-generic `mdspan` algorithm libraries.

As with the C++26 parallel ranges algorithms ([P3179R9]), only the non-parallel overloads are marked `constexpr`. We have no objection to marking the parallel overloads `constexpr` as long as this is done consistently for all the parallel algorithms.

§ 4.1 Iteration order

The key to this design is that `for_each_index` takes a layout mapping. This ensures that the implementation always gets an iteration order hint. Users are encouraged to access `mdspan` objects inside the loop in a way that the layout mapping’s suggested iteration order results in best performance.

We let different implementations choose different iteration orders for best performance. For example, architectures and programming models that favor coalesced access across neighboring threads may choose to parallelize over the contiguous extent. Architectures and programming models that favor spatial locality on a single thread (e.g., due to cache lines) may choose to parallelize over the opposite, “least contiguous” extent.

While the NVIDIA CUB library [CUB] only permits `layout_left` and `layout_right`, we plan on allowing any arbitrary layouts including custom layouts. Implementations can use information about the layout mapping’s strides or other properties (such as `is_exhaustive` and `is_unique`) in order to plan an optimal scheduling or iteration strategy. It’s worth noting that because of the unlimited customization possibilities of layout mappings, implementations may not always be able to pick the most optimal strategy.

§ 5 Design Considerations

§ 5.1 Should there be an integer overload for the rank-1 case?

We do not propose an overload of `for_each_index` that takes a single integer instead of a rank-1 extents object.

Our design does not exclude rank-1 extents. We deliberately leave in this case in order to support development of rank-generic `mdspan` algorithm libraries. Given that the rank-1 case works, it’s tempting to add a `for_each_index` overload that accepts an integer instead of an extents object.

```
int N = /* ... */;
auto f = [] (int x) { /* ... */ };
for_each_index(N, f);
```

This would act just like `ranges::for_each(views::indices(N), f)`. There are a couple issues with this overload.

1. It does not depend on extents at all, so there is no reason to put it in an `mdspan`-related header or module. If it exists, it should go in the `<algorithm>` header.
2. It’s not clear to users whether this is just a convenient abbreviation, or whether it is a basis operation. We do not want to claim an overload that could be useful for later addition of a basis operation.

The term “basis operation” arose in discussions and proposals leading up to `std::execution`, for example in P0443. In the context of parallel execution, WG21 understands it to mean

“an operation that would be used to implement other operations, and on which those operations depend for their performance.” The bulk sender adapter in `std::execution` evolved as a basis operation for asynchronous parallel algorithms. That’s why it accepts an integer loop bound, not a general range.

A `for_each_index` algorithm that takes an integer loop bound would thus strongly suggest a basis operation analogous to bulk. We do not wish to claim this design space. The point of this proposal is multidimensional iteration. The rank-1 case only exists to simplify development of generic `mdspan` algorithm libraries.

§ 5.2 Should there be extents overloads?

`CUB[CUB]` provides two overload sets: one taking a layout mapping and iterating over its extents, and one taking just an extents object. The analog of the latter would look like this.

```
std::for_each_index(std::execution::par_unseq,
    std::dims<3>(x, y, z),
    [=] (auto i, auto j, auto k) {
        B[j, i, k] = A[i, j, k];
    });
```

The default iteration direction would be implementation-defined.

We chose not to include extents overloads in order to ensure preservation of the spatial locality hint. If we had extents overloads, users might learn the pattern `for_each_index(x.extents(), f)`. That would work, but it would throw away information about spatial locality that the user already has. Forcing users to say `for_each_index(x.mapping(), f)` would not add verbosity, but it would preserve the hint.

If users have an `mdspan` over which they want to iterate, then they already have a layout mapping. Otherwise, users can create one on the fly.

```
std::for_each_index(std::execution::par_unseq,
    std::layout_right::mapping(std::dims<3>(x, y, z)),
    [=] (auto i, auto j, auto k) {
        B[j, i, k] = A[i, j, k];
    });
```

§ 5.3 Why permit custom layouts?

Current practice limits how users can specify or hint at iteration order. For example, `CUB[CUB]` only permits `layout_left` and `layout_right`. `OpenACC` and `OpenMP` permit permutation of loop index orders and have tiling options, but do not take arbitrary layouts.

One effect of `for_each_index` taking a layout mapping is there is no portable way for an implementation to recognize that users want to permute loop index orders or tile the loop. Users can write a custom layout mapping that has permuted loop indices and/or specifies tiling, but implementations would have no way to optimize for that.

We consider this acceptable, because the current “one-dimensional” parallel algorithms have no way to specify a “tiling.” Perhaps the right way to do tiling would look more like “SIMD algorithms” that iterate over tile values instead of a single value.

§ 5.4 Why not iterate over slices?

Users can call `submdspan` to get an `mdspan` that expresses a slice, and then iterate over the extents of that resulting `mdspan`.

If users just have a layout mapping without an `mdspan`, they can slice it by calling `submdspan_mapping` on it.

§ 5.5 Header name

We choose `<mdalgorithm>` as a header for the multidimensional `for_each_index` overloads, for two reasons.

1. It retains the `<mdspan>` naming convention (where “md” is a prefix for multidimensional things).
2. It keeps the `<mdspan>` header free from what might be a heavyweight implementation of algorithms that uses different parts of the Standard Library.

LEWG review of [P3242R1] in 2026-03 asked for the `mdspan` copy and fill algorithms to go in the `mdspan` header. LEWG might not have realized how substantial the implementations of those algorithms can be. They amount to several thousand lines of code in the Kokkos library, for example. Different hardware vendors such as NVIDIA offer lower-level multidimensional array copy interfaces that implementations could use. However, putting all that in the `<mdspan>` header might discourage users from using `mdspan` as an interface-layer type. For the same reason, we do not want to include `for_each_index` in the `<mdspan>` header. We also do not want to put the new algorithms in `<algorithm>`, because that would expose existing `<algorithm>` users to `mdspan`.

As we propose more multidimensional algorithms for the Standard Library, the motivation for a separate “multidimensional algorithms header” will only increase.

§ 5.6 Alternative Names

- `for_each_in_extents` for extents
 - As discussed elsewhere in this proposal, we do not propose extents overloads of `for_each_index`.
- `for_each_in_layout_mapping` for layout mappings
 - Even if we were to propose a separate algorithm that takes extents, we would see no need to differentiate it from the algorithm that takes a layout mapping.

The mapping is purely a hint. Iterating over a mapping unambiguously means iterating over its extents.

- `for_each`
 - Our algorithm’s input is not a range. (We talk about why that is below.)
 - We wish to avoid confusion with `ranges::for_each` or with legacy `std::for_each`.
 - We do not have the analog of the `ranges::for_each` overload that takes an (iterator, sentinel) pair.

§ 5.7 Should we make layout mappings and extents ranges and just use `ranges::for_each`?

No. Here are three reasons.

1. Ranges flatten out multidimensional information. This hinders checking for error cases like attempting to do `std::ranges::copy` from a 2 x 3 x 5 `mdspan` to a 6 x 5 `mdspan`.
2. Ranges would let users run `ranges::copy` or `ranges::transform` between `mdspans` with different layouts. That would, in turn, force the implementation to pick a single canonical iteration order (as otherwise the results would be incorrect).
3. Ranges makes views opaque. This loses multidimensional information that could be used to optimize.

Consider `std::ranges::copy` between two `mdspan` objects with different layouts. If we choose the layout mapping’s preferred iteration order, the copy would give incorrect results. If we choose a canonical iteration order (e.g., whatever `layout_right` would do), then the copy might perform badly unless implementations have a complicated specialization system that turns the range back into an `mdspan`.

In general, Ranges’ design makes views opaque. This hinders the ability of Standard Library implementations (or of users) to recognize and optimize for special cases of view expressions. Portable C++ code can see that a range is a specialization of `cartesian_product_view` of `iota_view`, but it can’t get the actual loop bounds. It can see that a range is a `cartesian_product_view` piped to a `transform_view`, but it can’t get the function out of the `transform_view`. This means that a Standard Library implementation that only uses portable features of Standard views can’t optimize Ranges iteration over a multidimensional index space. Implementations *could* add nonportable hooks to get multidimensional information back out of flattened-out views. However, that would hinder compiler-level optimizations (such as lowering to use OpenACC or OpenMP loops), because some compilers need to be able to build with different Standard Library implementations (e.g., both `libstdc++` and `libc++`).

§ 5.8 Does this supersede the need for a `for_each` (element) algorithm for `mdspan`?

No. There is still value in having a special algorithm to iterate over the elements of a single `mdspan`. For example, this enables optimization for known layouts and known accessors.

We showed an implementation of `elements_range` above, a view (in the Ranges sense) of the elements of an `mdspan`. Here is a more optimized implementation. Note the special cases for always exhaustive layout mappings and for the two “raw array accessors” `default_accessor` and `aligned_accessor`.

```
template<class T>
constexpr bool is_pointer_accessor_v = false;
template<class ElementType>
constexpr bool is_pointer_accessor_v<
    std::default_accessor<ElementType>> = true;
template<class ElementType, size_t ByteAlignment>
constexpr bool is_pointer_accessor_v<
    std::aligned_accessor<ElementType, ByteAlignment>> = true;

template<class ElementType, class Extents, class Layout, class Accessor>
constexpr auto
elements_range(std::mdspan<ElementType, Extents, Layout, Accessor> x)
    pre(x.is_unique())
{
    if constexpr (decltype(x)::is_always_exhaustive()) {
        size_t size = /* product of the extents of x */;
        if constexpr (is_pointer_accessor_v<Accessor>) {
            auto x_ptr = x.data_handle();
            return std::views::indices(size) |
                std::views::transform([x_ptr] (size_t k) {
                    return x_ptr[k];
                });
        }
        else {
            auto handle = x.data_handle();
            auto acc = x.accessor();
            return std::views::indices(size) |
                std::views::transform([handle, acc] (size_t k) {
                    return acc.access(handle, k);
                });
        }
    }
    else {
        return extents_range(x.extents()) | std::views::transform(
            [x] (auto index_tuple) {
                auto [...indices] = index_tuple;
                return x[indices...];
            });
    }
}
```

§ 5.9 Do we want to return anything?

No. There is no need for an `mdspan`-based `for_each_index` to return anything.

§ 5.9.1 No need to return an “iterator” analog

Ranges algorithms return iterators for two reasons.

1. Their arguments might be single-pass ranges (like `input_range`).
2. The algorithm might not have used up the whole range, and users want to know where it stopped.

The `mdspan` class has a “multipass” guarantee; it behaves like the multidimensional analog of a forward range. Furthermore, if `for_each_index` returns normally (not through an exception thrown by `mdspan` or the callable), then there is no reason for it not to visit the entire domain.

Even if an `mdspan`-based algorithm had a way to return normally without visiting the entire domain, it wouldn’t make sense to return the analog of “one past the last visited multidimensional index,” because that would presume a particular iteration order. Otherwise, how would callers know the “next” multidimensional index after the returned one?

§ 5.9.2 No need to return the function object

Only nonparallel ranges::`for_each` returns a function object. This makes sense because the algorithm iterates in a given order, so the input function object could meaningfully accumulate state. Also, for the nonparallel version it’s not necessary to copy the input function object. (Parallel algorithms reserve the right to copy the function object in order to avoid sharing between threads, and also to permit accelerator-based implementations that do the equivalent of `memcpy` to put their arguments in the accelerator’s memory.)

Our proposed `for_each_index` does not iterate in a specified order. This makes accumulation of state in a function object less meaningful.

§ 6 Proposed Wording

§ 6.1 Add `__cpp_lib_for_each_index` macro

[Editor's note: In [version.syn], add the `__cpp_lib_for_each_index` feature test macro definition. Adjust the placeholder value as needed so as to denote this proposal’s date of adoption.]

```
#define __cpp_lib_flat_set 202511L // also in  
    <flat_set>
```

```
#define __cpp_lib_for_each_index 202511L // also in  
    <mdalgorithm>
```

```
#define __cpp_lib_format 202311L // also in
    <format>
#define __cpp_lib_format_path 202506L // also in
    <filesystem>
```

[Editor's note: Add the following to 26 [algorithms].]

§ 26.?? Header `<mdalgorithm>` synopsis

```
namespace std {

template<class Mapping, class Fun>
constexpr void for_each_index(const Mapping& mapping, Fun fun);

template<class ExecutionPolicy, class Mapping, class Fun>
void for_each_index(ExecutionPolicy&& policy, const Mapping& mapping, Fun
    fun);

}
```

§ 26.?? Multidimensional algorithms

§ 26.??1 For each index

- 1 Multidimensional for each index algorithms iterate over the extents of a given layout mapping. The mapping hints at an iteration order with good performance properties.

```
template<class Mapping, class Fun>
constexpr void for_each_index(const Mapping& mapping, Fun fun);

template<class ExecutionPolicy, class Mapping, class Fun>
void for_each_index(ExecutionPolicy&& policy, const Mapping& mapping, Fun
    fun);
```

- 2 *Constraints:*

- (2.1) — Mapping meets the layout mapping requirements (23.7.3.4.2 [[mdspan.layout.reqmts](#)]).
- (2.2) — Fun meets the `copy_constructible` requirements.
- (2.3) — Fun meets the `invocable<F&, IT...>` requirements, where IT is a pack of `Mapping::rank()` instances of `Mapping::index_type`.

- 3 *Effects:* Calls `invoke(f, i...)` for every multidimensional index `i` in `mapping.extents()`.

- 4 *Complexity:* Applies fun exactly the size of `mapping.extents()` times.

- 5 *Remarks:* If fun returns a result, the result is ignored.

- 6 *Recommended practice:* Implementations are encouraged to iterate the multidimensional index space in an order that would be expected to perform well for iteration over `mdspan` objects with the given execution policy and layout mapping.

§ 7 Acknowledgments

Sandia National Laboratories is a multimission laboratory managed and operated by National Technology & Engineering Solutions of Sandia, LLC, a wholly owned subsidiary of Honeywell International Inc., for the U.S. Department of Energy's National Nuclear Security Administration under contract DE-NA0003525. This paper describes objective technical results and analysis. Any subjective views or opinions that might be expressed in the paper do not necessarily represent the views of the U.S. Department of Energy or the United States Government.

§ 8 References

[CUB] cub::DeviceFor.

https://nvidia.github.io/cccl/unstable/cub/api/structcub_1_1DeviceFor.html

[HPX] hpx::experimental::for_loop.

https://hpx-docs.stellar-group.org/latest/html/libs/core/algorithms/api/for_loop.html

[Kokkos] Kokkos.

<https://kokkos.org/kokkos-core-wiki/API/core/policies/MDRangePolicy.html>

[OpenACC] OpenACC.

<https://www.openacc.org/>

[OpenMP] OpenMP.

<https://www.openmp.org/>

[P0009R18] Christian Trott, D.S. Hollman, Damien Lebrun-Grandie, Mark Hoemmen, Daniel Sunderland, H. Carter Edwards, Bryce Adelstein Lelbach, Mauro Bianco, Ben Sander, Athanasios Iliopoulos, John Michopoulos, Nevin Liber. 2022-07-13. MDSPAN.

<https://wg21.link/p0009r18>

[P2164R9] Corentin Jabot. 2022-12-07. views::enumerate.

<https://wg21.link/p2164r9>

[P3060R3] Weile Wei, Zhihao Yuan. 2025-06-19. Add std::views::indices(n).

<https://wg21.link/p3060r3>

[P3179R9] Ruslan Arutyunyan, Alexey Kukanov, Bryce Adelstein Lelbach. 2025-05-29. C++ parallel range algorithms.

<https://wg21.link/p3179r9>

[P3242R1] Nicolas Morales, Christian Trott, Mark Hoemmen, Damien Lebrun-Grandie. 2025-03-13. Copy and fill for mdspan.

<https://wg21.link/p3242r1>